

Le jeu des tuiles

Barchin et Charos jouent à un jeu sur une grille de $2N \times 2M$ cellules. Les lignes sont numérotées 0 à $2N - 1$ de haut en bas, et les colonnes sont numérotées 0 à $2M - 1$ de gauche à droite. Pour $0 \leq i < 2N$ et $0 \leq j < 2M$, nous désignons la cellule de la ligne i et de la colonne j par (i, j) .

Barchin donne à Charos $N \cdot M$ **blocs**, un par un. Chaque bloc est un carré de 2×2 composé de quatre **tuiles** 1×1 . Barchin colore chaque tuile du bloc en noir ou en blanc, tout en garantissant qu'**au moins une tuile est blanche**.

Charos doit placer chaque bloc sur la grille **immédiatement après l'avoir reçu**, sans savoir quels blocs elle recevra plus tard. Les blocs ne peuvent pas être tournés. Chaque bloc doit être placé entièrement à l'intérieur de la grille, couvrant exactement quatre cases. De plus, la case en haut à gauche de chaque bloc doit recouvrir une case dont les coordonnées de ligne et de colonne sont paires. Chaque cellule de la grille peut être recouverte par au plus un bloc.

Barchin gagne la partie si, après la pose d'un bloc, il existe un carré de 2×2 cases recouvert de quatre tuiles noires. Plus précisément, si les cases (a, b) , $(a + 1, b)$, $(a, b + 1)$ et $(a + 1, b + 1)$ sont toutes recouvertes de tuiles noires pour certaines valeurs de $0 \leq a < 2N - 1$ et $0 \leq b < 2M - 1$, alors Barchin gagne. Les indices a et b peuvent être pairs ou impairs.

Charos gagne si elle place les $N \cdot M$ blocs sans qu'à aucun moment Barchin ne gagne. Notez que le placement de $N \cdot M$ blocs couvrira complètement la grille.

Votre tâche consiste à mettre en œuvre une stratégie permettant à Charos de gagner la partie. Il est possible de démontrer que, sous les contraintes données, Charos peut toujours placer les blocs de manière à se garantir une victoire, quelles que soient les couleurs des blocs qu'elle recevra ultérieurement.

Détails d'implémentation

Vous devez implémenter deux fonctions :

```
void init(int N, int M)
```

- N : la moitié du nombre de lignes dans la grille.
- M : la moitié du nombre de colonnes dans la grille.

- La fonction est appelée exactement une fois par fichier test, au début de l'exécution de votre programme.

```
std::pair<int, int> receive_block(int TL, int TR, int BL, int BR)
```

- TL, TR, BL, BR : les couleurs respectives des cases en haut à gauche, en haut à droite, en bas à gauche et en bas à droite du bloc actuel, comme illustré ci-dessous. Chaque valeur est soit 0 (blanc), soit 1 (noir).
- Cette fonction est appelée exactement $N \cdot M$ fois par test, après l'appel initial à `init`.



Cette fonction doit renvoyer une paire d'entiers (i, j) , où i est la ligne et j est la colonne de la cellule où la tuile en haut à gauche de ce bloc doit être placée. À la fois i et j doivent être pairs, et la région 2×2 couverte par le bloc ne doit pas chevaucher un bloc précédemment placé.

Si `receive_block` renvoie une paire qui ne satisfait pas ces exigences ou si, après avoir placé le bloc, un carré de 2×2 cellules est complètement recouvert de tuiles noires, l'évaluateur arrêtera immédiatement votre programme et le verdict pour le fichier test sera `Output isn't correct`.

Le comportement du correcteur n'est **pas adaptatif**. Cela signifie que la séquence de blocs que Barchin transmet à Charos est fixé avant l'appel à `init`.

Contraintes

Pour chaque bloc, soit S le nombre de tuiles noires parmi ses quatre tuiles, c'est-à-dire, $S = TL + TR + BL + BR$.

- $1 \leq N, M \leq 100$
- $0 \leq S \leq 3$ pour chaque bloc.

Sous-tâches

Sous-tâche	Score	Contraintes supplémentaires
1	6	$S = 1$ pour chaque bloc, et $N = 2$.
2	16	$S = 3$ pour chaque bloc. $N = M$, N est pair, et chacune des quatre combinaisons de blocs possibles apparaît exactement $\frac{N^2}{4}$ fois.
3	10	$S = 1$ pour chaque bloc.
4	29	$S \leq 2$ pour chaque bloc.
5	39	Aucune contrainte supplémentaire.

Exemple

Considérons un jeu avec $N = 1$ et $M = 2$: la grille a 2 lignes et 4 colonnes. L'évaluateur appelle en premier :

```
init(1, 2)
```

Au départ, toutes les cellules sont vides. La grille ressemble à ceci :

	0	1	2	3
0				
1				

Il y a $N \cdot M = 2$ blocs à placer. Supposons que Barchin donne un bloc avec trois tuiles noires et une tuile blanche dans le coin supérieur droit. L'évaluateur appelle :

```
receive_block(1, 0, 1, 1)
```

Charos décide de placer ce bloc sur le côté gauche de la grille en renvoyant $(0, 0)$.

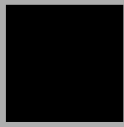
La grille ressemble maintenant à ceci :

	0	1	2	3
0				
1				

Barchin propose ensuite un autre bloc avec trois tuiles noires et une tuile blanche dans le coin supérieur gauche :

```
receive_block(0, 1, 1, 1)
```

La seule cellule restante avec une ligne et une colonne paires pouvant servir de coin supérieur gauche d'un bloc 2×2 est $(0, 2)$, donc Charos renvoie $(0, 2)$. La grille se présente alors comme suit :

	0	1	2	3
0				
1				

Aucun carré 2×2 n'est entièrement recouvert de tuiles noires ; Charos a donc placé tous ses blocs sans que Barchin ne puisse gagner. Charos remporte la partie.

Évaluateur d'exemple (grader)

Format d'entrée :

```
N M
TL[0] TR[0] BL[0] BR[0]
TL[1] TR[1] BL[1] BR[1]
...
TL[NM-1] TR[NM-1] BL[NM-1] BR[NM-1]
```

Format de sortie :

```
R[0] C[0]  
R[1] C[1]  
...  
R[NM-1] C[NM-1]
```

Ici, $R[k]$ et $C[k]$ sont la paire d'entiers renvoyés par le k -ième appel à `receive_block`.